

Scientific Computing & Advanced Programming Lab

SAURAV SAMANTARAY

Department of Mathematics
IIT Madras

Lecture-Slides-3



sauravsrav@iitm.ac.in

From Data Structures to Objects

- ▶ So far, we have seen different ways of organising data.
- ▶ For example, a point can be represented using a tuple:
 $P = (3, 4)$
- ▶ We can store the data, but what about operations associated with that data?
- ▶ For a point, we might want to:
 - ▶ Find its distance from the origin.
 - ▶ Find its distance from another point.
 - ▶ Translate the point.
 - ▶ Find its polar coordinates.
- ▶ Instead of keeping the **data** and the **functions** separate, we can combine them into a single entity.

Data + Operations	→	Object
-------------------	---	--------



Introduction to Object-Oriented Programming

- ▶ **Object-Oriented Programming (OOP)** organizes a program around **objects**.
- ▶ An object contains:
 - ▶ **Attributes** — data describing the object.
 - ▶ **Methods** — operations that the object can perform.

Example: A Point

Point
x, y
distance() translate() polar_coordinates()

- ▶ x and y are the **attributes**,
- ▶ the functions are the **methods**.

The object therefore combines the **state** of an entity with the **operations** that act on that state.



Classes and Objects

- ▶ A **class** is a blueprint for creating objects.
- ▶ An **object** is a particular instance of a class.
- ▶ The class defines what a `Point` contains and what it can do.
- ▶ We can then create different objects:

```
P = Point(3, 4)
```

```
Q = Point(7, 2)
```

Point class $\longrightarrow$ blueprint



$P = (3, 4)$

$Q = (7, 2)$

One class $\longrightarrow$ many objects



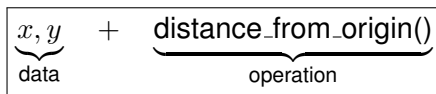
A First Mathematical Class: `Point`

- ▶ We want a Python object to represent a point $P = (x, y)$.
- ▶ The point should store its coordinates and calculate its distance from the origin.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def distance_from_origin(self):
        return (self.x**2 + self.y**2)**0.5
```

The class combines:



Using the `Point` Class

- ▶ The class is a blueprint. We can create different **objects** from it.

```
P = Point(3, 4)
Q = Point(6, 8)
```

- ▶ Each object has its own coordinates:

```
print(P.x, P.y)      # 3 4
print(Q.x, Q.y)      # 6 8
```

- ▶ The same method works on different objects:

```
print(P.distance_from_origin()) # 5.0
print(Q.distance_from_origin()) # 10.0
```

One class → many independent objects
--



Dissecting the Point Class

```
class Point:
```

- ▶ Defines a new type called `Point`.

```
def __init__(self, x, y):  
    self.x = x  
    self.y = y
```

- ▶ `__init__` initializes a new object.
- ▶ `x` and `y` are input parameters.
- ▶ `self` refers to the particular object.
- ▶ `self.x` and `self.y` are object attributes.

```
def distance_from_origin(self):  
    return (self.x**2 + self.y**2)**0.5
```

- ▶ A function defined inside a class is called a **method**.
- ▶ `self.x` and `self.y` refer to the coordinates of the



Exercise: Extend the `Point` Class

- We have the `point` class defined as before

Task: Add the following features

1. Add a method `distance_to(other)` that calculates the distance between two points.
2. Add a method `translate(dx, dy)` that moves the point by

$$(x, y) \longrightarrow (x + dx, y + dy).$$

3. Add a method `midpoint(other)` that returns the midpoint between two points.

Test your class with

```
P = Point(2, 3)
```

```
Q = Point(8, 7)
```

